

Implementación de una red de comunicación RS-485 con arquitectura maestro-esclavo utilizando ESP-32

Emiliano Villarreal, Sabrina Higgimbotton, Roberto M. Murdocca, Sergio F. Hernandez Velazquez
Departamento de Electrónica
Universidad Nacional de San Luis
San Luis, Argentina
e-mail: {emiliano02villarreal, sabrinahiggimbotton, mmurdocc, sergio.sfhv}@gmail.com

Resumen - El presente proyecto describe el diseño e implementación de una red de comunicación RS-485 con arquitectura maestro-esclavo, orientado a su aplicación en entornos de enseñanza dentro de la materia Interfaces. El sistema incluye un maestro ESP-32 que controla tres esclavos mediante tramas de 8 bits y permite seleccionar funciones específicas mediante un menú visual en un display. Los esclavos, también basados en ESP-32, incorporan sensores y actuadores: un sensor DHT-11, un sensor ultrasónico HC-SR04, un buzzer, un motor SG90 y LEDs. Se logra una comunicación Half-Duplex confiable y se implementan lógicas no bloqueantes con `millis()` para evitar interrupciones, lo que permite una interacción fluida en el sistema.

I. INTRODUCCIÓN

La norma RS-485 es ampliamente utilizada en sistemas industriales por su robustez, bajo costo y capacidad para conectar múltiples dispositivos en un mismo bus. Estas características la hacen ideal para su incorporación en entornos educativos. Este trabajo presenta una red de comunicación basada en RS-485 con topología maestro-esclavo, compuesta íntegramente por módulos ESP32. El desarrollo se enmarca en la materia Interfaces y tiene como objetivo facilitar el aprendizaje de conceptos clave como la comunicación Half-Duplex, direccionamiento de dispositivos y diseño de protocolos simples. El sistema incorpora sensores y actuadores distribuidos, permitiendo prácticas didácticas de control e interacción en red.

II. ARQUITECTURA DEL SISTEMA

A. Maestro

El dispositivo maestro está implementado con un ESP32 que transmite tramas de 8 bits por el bus RS-485, tal como se muestra en la Figura 1. Cada trama está compuesta por:

- 2 bits de dirección del esclavo,
- 3 bits para la función solicitada,
- 3 bits de CRC (constante en esta implementación).



Figura 1: Trama enviada por el Master.

Dicha trama se genera utilizando la función "crearTrama", a la cual se le deben pasar como parámetros el id del esclavo al que se quiere comunicar, la función elegida y el CRC.

El sistema permite la selección de esclavos y funciones mediante tres botones (↑, ↓ y OK) y presenta la información en un display LCD I2C de 16x2 ubicado en el master.

B. Esclavos

Cada esclavo posee una dirección única y responde únicamente a las tramas que le corresponden. Están distribuidos de la siguiente manera:

- Esclavo 1: Encendido de un LED o lectura de sensor DHT11 (temperatura y

- humedad).
- Esclavo 2: Control del sensor HC-SR04 o activación de un buzzer.
- Esclavo 3: Control de un servo motor SG90 (barrido de 0° a 180° y retorno), o encendido de LED.

El firmware de cada esclavo fue diseñado para recibir y decodificar la trama enviada por el maestro, ejecutar la acción correspondiente y, si es necesario, envía una respuesta al maestro a través del mismo bus RS-485.

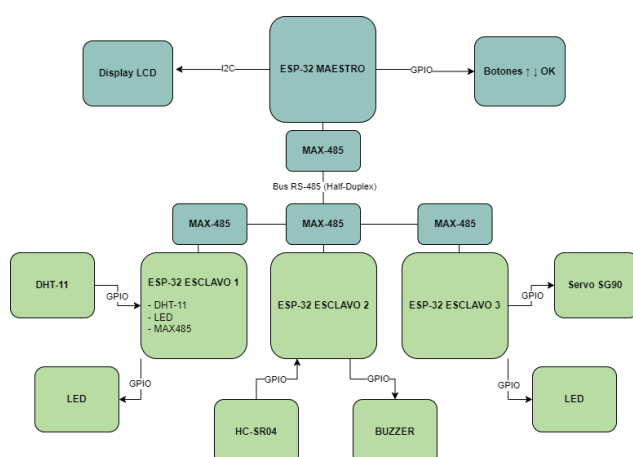


Figura 2: Diagrama en bloque de la red.

III. DISEÑO DEL SISTEMA

A. Comunicación

La comunicación entre nodos se realiza mediante módulos MAX-485, que permiten la conversión de niveles TTL a RS-485. El sistema opera en modo Half-Duplex, lo cual implica que la transmisión y recepción se alternan utilizando el mismo par de cables. Dado que el entorno de uso es un laboratorio educativo, no se empleó un cableado trenzado diferencial (A/B), ya que las distancias son reducidas y no se requieren medidas adicionales para mitigar interferencias.

B. Protocolo

RS-485 es un estándar de comunicación serie definido por la norma EIA/TIA-485, utilizado ampliamente en entornos industriales y de automatización por su robustez frente al ruido y su capacidad multipunto.

Ventajas principales:

- Comunicación diferencial, lo que incrementa la inmunidad al ruido.
- Hasta 32 transmisores y 32 receptores conectados en un mismo bus (extensible con repetidores).
- Alcance de hasta 1200 metros a bajas velocidades.
- Velocidades de hasta 10 Mbps en distancias cortas.
- Posibilidad de operar en modo Half-Duplex o Full-Duplex (requiere dos pares de cables).

Resistencias de Terminación

Una red RS-485 bien terminada requiere resistencias cuya impedancia coincida con la del cableado utilizado, lo que evita reflexiones y distorsiones en la señal. Sin embargo, para redes cortas y de baja velocidad —como este caso educativo—, es común prescindir de estas resistencias para simplificar la implementación. Este diseño funciona correctamente sin terminaciones, dado que los cables no superan los 100 metros y la velocidad de transmisión es inferior a 19200 bps.

C. Lógica no bloqueante

Para evitar que un dispositivo monopolice el bus, todas las rutinas de espera fueron reemplazadas por código no bloqueante basado en `millis()`. Esto fue particularmente importante, ya que al utilizar delays se producían errores de sincronización.

IV. DISEÑO Y METODOLOGÍA

El diseño del sistema se planteó desde una lógica modular, dividiendo el conjunto en cuatro nodos principales (maestro y tres esclavos), cada uno con funciones específicas pero conectados bajo un mismo bus RS-485. Se optó por el microcontrolador ESP-32 en todos los módulos, lo cual permitió un entorno de

programación unificado y aprovechó sus múltiples puertos UART.

Para la transmisión de datos, se optó por la trama antes mencionada, de 8 bits, la cual por su simplicidad en el protocolo tiene como fin facilitar la comprensión del sistema por parte de estudiantes en prácticas de laboratorio. La selección de sensores y actuadores también estuvo orientada a lograr un sistema interactivo y didáctico.

El maestro fue diseñado con una interfaz de usuario basada en tres botones físicos que permiten navegar por un menú visible en un display LCD I2C. La elección de esta interfaz busca simular un entorno de control real, donde el usuario selecciona dispositivos y funciones de forma directa.

En los esclavos, se implementó una estructura que permite responder a los comandos del maestro de manera no bloqueante, evitando el uso de `delay()` y priorizando el control a través de `millis()`. Esta decisión fue fundamental para mantener la red activa y receptiva, aún durante la ejecución de funciones prolongadas como el movimiento del servo o la reproducción de un tono en el buzzer.

V. IMPLEMENTACIÓN

Esta sección describe los componentes físicos y lógicos utilizados en el desarrollo del sistema, incluyendo su configuración y función dentro de la red.

A. Hardware

Cada nodo del sistema cuenta con un microcontrolador ESP-32 y un transceptor MAX-485 para convertir los niveles TTL a RS-485. El maestro y los esclavos están alimentados mediante una fuente regulada de 5 V conectada al pin Vin de cada ESP32.

Pantalla LCD I2C

Para la visualización de información en el nodo

maestro se emplea una pantalla LCD 16x2 con interfaz I2C. Esta interfaz permite reducir la cantidad de pines requeridos, facilitando el cableado y la programación.

Botonera de Navegación

El sistema incluye tres botones conectados a los pines GPIO 13 (↑), GPIO 12 (↓) y GPIO 14 (OK) del ESP32 maestro. Se implementó un control de rebote por software utilizando la librería Bounce2, garantizando una lectura estable de las pulsaciones.

Sensores y Actuadores

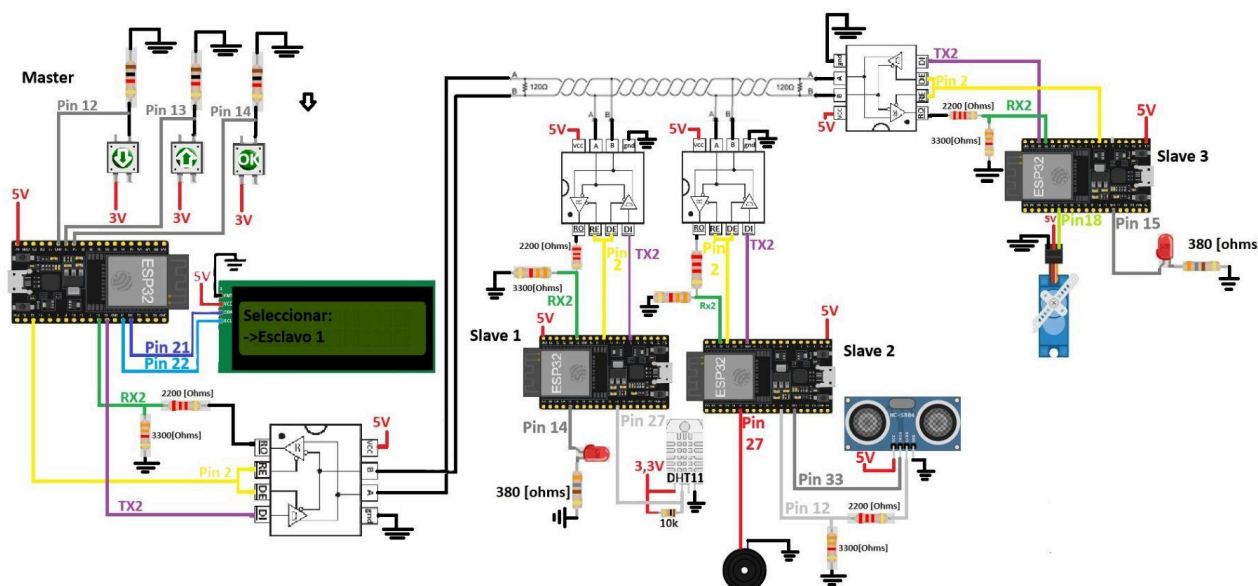
Cada esclavo se encarga de controlar o leer distintos dispositivos periféricos:

- **Esclavo 1**

- **LED:** Encendido mediante comando del maestro.
- **Sensor DHT11:** Mide temperatura y humedad, y transmite los datos al maestro.

- **Esclavo 2**

- **Sensor HC-SR04:** Sensor ultrasónico para medición de distancia.
- **Buzzer:** Activado remotamente mediante comando.



• Esclavo 3

Servo motor SG90: Ejecuta un barrido angular controlado por el maestro.

- **LED adicional:** Activación simple desde el nodo maestro.

- En caso de lectura, responden al maestro por el mismo bus.

VI. Resultados

El sistema fue probado en un entorno de laboratorio, donde se validó su funcionamiento completo con todos los nodos conectados simultáneamente. Se obtuvieron los siguientes resultados:

- Los esclavos respondieron correctamente a cada función solicitada por el maestro.
- Lectura fiable del sensor DHT11 (temp/hum) y HC-SR04 (distancia).
- Activación controlada del buzzer, barrido del motor SG90 y encendido de LEDs.

B. Software

El código fue desarrollado en C++ utilizando el entorno de programación de Arduino.

Estructura del Software

El maestro contiene funciones para:

- Navegar el menú mediante botones.
- Mostrar información en pantalla.
- Generar tramas y enviarlas por el bus RS-485.
- Recibir respuestas de los esclavos y actualizarlas en el display.

Los esclavos ejecutan rutinas que:

- Reciben y decodifican la trama.
- Ejecutan la acción correspondiente según la función y dirección.

VI. Conclusiones

El sistema desarrollado demuestra una implementación eficiente y funcional de una red de comunicación RS-485 utilizando microcontroladores ESP-32. Gracias a su arquitectura distribuida, el nodo maestro puede controlar múltiples dispositivos esclavos, cada uno con funciones específicas, como la lectura de sensores y el control de actuadores. La integración de una pantalla LCD I2C y botones de navegación facilita la interacción del usuario, permitiendo seleccionar nodos y visualizar información de manera clara. Este

diseño modular no solo garantiza robustez y escalabilidad, sino que también sirve como una excelente base para aplicaciones más complejas en automatización, monitoreo ambiental y control industrial.

VIII.REFERENCIAS

- [1] ESP32 Technical Reference Manual - Autor: Espressif Systems
- [2] RS-485 Basics Series - Autor: Texas Instruments
- [3] CONCEPTOS FUNDAMENTALES DE RS485 Y RS422 - Autor: NOVUS
- [4] Guía de la comunicación RS485 - Autor: Electronic team, inc.
- [5] DHT11 Datasheet - Autor: Aosong Electronics
- [6] SG90 Servo Motor Datasheet - Autor: Tower Pro
- [7] PWM Control of SG90 Servo with ESP32 - Autor: How To Mechatronics
- [8] MAX-485 Datasheet - Autor: Maxim Integrated